

JB Python

Test cover subtitle

Table of Contents

1. default	6
2. Help	7
3. Links	8
4. pip	9
5. Py	10
6. IDE	11
6.1. Jupyter	11
6.2. VS Code (Visual Studio)	11
6.2.1. Setup	11
6.2.2. Debug	12
6.2.3. Control Keys	12
6.2.4. Program Control	13
6.2.4.1. Run Code	13
6.2.4.2. Initialize	13
6.2.5. Jupyter in VScode	13
6.2.6. Zen Mode	13
6.2.7. Terminal	13
6.2.8. extensions	13
6.3. PyCharm	13
6.3.1. Links	14
6.3.2. New Project	14
6.3.3. Refactoring	14
6.3.4. Shortcuts	14
6.4. Visual Studio	14
6.5. Komodo Edit	14
7. Syntax	15
7.1. Reserved Words	15
7.2. Import options	15
8. Datatypes	16
8.1. Numerical Types	16
8.1.1. Math Functions	16
8.1.1.1. Math module	16
8.2. String Types	16
8.2.1. String formatting	17
8.2.2. Methods	18
8.2.2.1. capitalize	18
8.2.2.2. casefold	18
8.2.2.3. center	18
8.2.2.4. encode	18
8.2.2.5. endswith	19
8.2.2.6. expandtabs	19
8.2.2.7. find	19
8.2.2.8. format	20
8.2.2.9. format.map	21
8.2.2.10. index	21
8.2.2.11. isalnum	21
8.2.2.12. isalpha	21
8.2.2.13. isascii	22

8.2.2.14. isdecimal	22
8.2.2.15. isdigit	22
8.2.2.16. isidentifier	22
8.2.2.17. islower	23
8.2.2.18. isnumeric	23
8.2.2.19. isprintable	23
8.2.2.20. isspace	23
8.2.2.21. istitle	23
8.2.2.22. isupper	24
8.2.2.23. join	24
8.2.2.24. ljust	24
8.2.2.25. lower	24
8.2.2.26. lstrip	25
8.2.2.27. maketrans	25
8.2.2.28. partition	25
8.2.2.29. replace	25
8.2.2.30. rfind	26
8.2.2.31. rindex	26
8.2.2.32. rjust	26
8.2.2.33. rpartition	26
8.2.2.34. rsplit	26
8.2.2.35. rstrip	27
8.2.2.36. split	27
8.2.2.37. splitlines	27
8.2.2.38. startswith	28
8.2.2.39. strip	29
8.2.2.40. swapcase	29
8.2.2.41. title	29
8.2.2.42. translate	30
8.2.2.43. upper	30
8.2.2.44. zfill	31
8.3. Bool	31
8.4. Collection Types	31
8.4.1. Lists	31
8.4.1.1. Strings	32
8.4.2. Tuples	33
8.4.3. Sets	33
8.4.3.1. Methods	34
8.4.4. Dictionaries	34
8.5. type conversion	35

9. Flow Control..... 36

9.1. if	36
9.2. for loop	36
9.3. while loop	37
9.4. range function	37
9.5. Loop Controls	37
9.6. pass	37

10. Operators..... 38

10.1. Arithmetic	38
10.2. Bitwise Operators	38
10.3. Comparison	38
10.4. Boolean Operators	38
10.5. Precedence	39

11. Howto 40

11.1. Sorting	40
11.2. Range	40
11.3. Enumerate	41
11.4. Zip	41
11.5. Input	42
11.6. *args	42
11.7. map	42

11.8. filter.....	42
-------------------	----

12. Functions..... 43

12.1. Definition	43
12.2. Arguments	43
12.3. Return	44
12.4. Scope	45
12.5. Built-in Functions	45
12.6. Lambda	47

13. Classes 48

13.1. Creation	48
13.2. Class Variables	48
13.3. Methods	49
13.4. Inheritance	50
13.5. Class Example	50

14. File I/O 52

14.1. File R/W	52
14.1.1. Try/Except	53
14.2. json	53
14.2.1. sample 1	53
14.2.2. sample 2	54
14.3. Operating system	54
14.3.1. getcwd.....	54
14.3.2. chdir	54
14.3.3. listdir.....	55
14.3.4. mkdir	55
14.3.5. rmdir.....	55
14.3.6. rename	55
14.3.7. stat	55
14.3.8. dirpath	55
14.3.9. walk	55
14.3.10. environ	56
14.3.11. path.exist.....	56
14.3.12. path.splitext.....	56
14.3.13. path.isdir	56
14.3.14. path.isfile	56

15. Librarys..... 57

15.1. Importing styles	57
15.1.1. Path	58
15.2. Standard Library	58
15.3. Time&Date	58
15.3.1. Time	58
15.3.2. DateTime	58
15.3.3. Calendar	59
15.4. Random	60
15.5. Regular Expressions	60
15.5.1. Misc.....	62
15.5.2. Functions	62
15.5.2.1. re.search	62
15.5.2.2. re.findall	62
15.5.2.3. re.compile	62
15.5.2.4. re,copy_reg	62
15.5.2.5. re.error	62
15.5.2.6. re.escape	62
15.5.2.7. re.finditer	62
15.5.2.8. re.match	63
15.5.2.9. re.purge.....	63
15.5.2.10. re.split	63
15.5.2.11. re.sre_compile.....	63

15.5.2.12. re.sub	63
15.5.2.13. re.subn	63
15.5.2.14. re.template	64
15.5.3. Examples	64
15.6. Matplotlib	64
15.7. Internet	65
15.7.1. Misc	65
15.7.2. Sockets	65
15.7.3. urllib	66
15.7.4. requests	67
15.7.5. Web Scraping	67
15.7.5.1. requests	67
15.7.5.2. Beautiful Soup	68
15.7.5.3. Requests-HTML	69
15.7.5.3.1. Functions	70
15.8. subprocess	70
15.9. BeautifulSoup4	71
15.10. numpy	71
15.11. Pandas	73
15.12. webbrowser	75
15.13. pyautogui	75
15.13.1. samples	75
15.14. Selenium	76
15.15. AutoIT	76
16. Functional Programming	78
17. Database	79
17.1. Install	79
17.2. Commands	79
18. Scratch	80
18.1. Spider	80
18.2. mac_changer	81
18.3. mac_changer1	82
18.4. mac_changer2	82

1. default

Should reinstall Python checking import debug symbols

2. Help

```
greeting= 'Hello'  
name = 'John'
```

```
import sys
```

dir = to see all commands available

```
dir(sys)
```

help() = to see info on command

```
help(sys)
```

```
help(sys.exit)
```

Could also do google search e.g. 'python sys.exit'

```
if __name__ == '__main__': main()
```

in linux

```
#!v/usr/bin/python3.7 -tt
```

```
-tt Changes any tab to spaces
```

Two ways to run a program from command line

```
python program.py
```

```
./program.py
```

3. Links

[Python 3.7.4 documentation](#)

[Real Python Tutorials](#)

Learning Python

Python Essential Training

Aptana studio

<https://forums.kjdelectronics.com/index.php?/forum/21-python-beginner-series/>

Source: [\[#\] Python Beginner Series - kjdElectronics Forums \(One Stop Programming\)](#) (18 June 2019 12:43 PM)

4. pip

```
pip help
pip help install
pip install requests-html
pip search package
pip list (lists all packages installed)
pip uninstall packagename
pip list -o (to see what files need updating)
pip install -U package
3:44
```

5. Py

6. IDE

6.1. Jupyter

[Anaconda Documentation](#)

[Jupyter Notebook User Guide](#)

6.2. VS Code (Visual Studio)

[Microsoft VS Code home page](#)

[Setup - Corey Shafer 1:15:16](#)

[Setup - Programming Knowledge 16:20](#)

[Visual Studio Code Intro & Setup](#)

[Getting Started with Python in Visual Studio Code](#)

[Getting Started with Python in Visual Studio Code](#)

[Visual Studio Code Web Dev Setup In 6 Minutes](#)

To open up command panel
ctl+Shift+p

To open program
Navigate to target directory and enter: code .

To reduce clutter
Click explorer button to toggle left paanel on and off
Zen mode (on and off)
^k z

terminal window
open: ctl backtic
select, add terminal, split, kill, expand, close
settings>terminal integrated: Has bunch of settings

REPL interpreter
Shift Enter: Runs selected lines

Better Comments tags: ! ? // todo *

to format code
shift alt f

6.2.1. Setup

Download Python and install with 'add to path box' checked

Lode python extention from Microsoft
install linter
install autopep8 - pip3 install autopep8

IronPython done with c#

6.2.2. Debug

debugging

have to add debugging configuration
launch.json for debugging
stop on entry

F5	Start debugging
Ctrl F5	Start without debugging
F9	Toggle Breakpoint
F10	Step - executed next line
F11	Step into
Shift F11	Step completely out of function



1. Continue (F5)
2. Step over (F10)
3. Step Into (F11)
4. Step out (Shift F11)
5. Restart
6. Stop

Breakpoints

Conditional breakpoints

1. Based on condition like `x>200`
2. Based on hits, i.e. number of times a line of code is executed.

Debugging parameters set in launch.json file which is a part of the .vscode in the working directory.

```
{
  "name": "Python: Current File",
  "type": "python",
  "request": "launch",
  "program": "${file}",
  "console": "integratedTerminal",
  "stopOnEntry": true
},
```

To specify command-line arguments for the Python program, add a line `"args": []` to the configuration, and place each argument as elements inside the `[]` list. For examples, see [Debugging - args](#).

6.2.3. Control Keys

alt up/down	Move line
alt shift down	Duplicate line or group
ctrl slash	Comment/uncomment line
ctrl `	Interactive Terminal screen

ctl shift m	problem panel
ctl shift p	command Pallet
ctl i then z	Zen Mode

6.2.4. Program Control

6.2.4.1. Run Code

Right click window and select run in terminal

1. With cursor on line press Shift Enter
2. Highlight section and press Shift Enter

6.2.4.2. Initialize

1. Make a directory in explorer, go there, and enter 'code .'
2. Then create file with .py extension

To create virtual environment
in terminal: `py -3 -m venv pictures`

6.2.5. Jupyter in VScode

6.2.6. Zen Mode

Elimantes all of screen except for the code that is being worked on

zen mode = `ctr i then z`
or `view>appearance>zen`

6.2.7. Terminal

Run selected code in terminal
by shift enter
search python formatting make sure is checked
search for 'format on save'

Terminal
Select in view menu or `ctr ``
Can run multiple terminals in split mode
terminal.integrated to set options

6.2.8. extensions

autodocstring
better comments
rope

6.3. PyCharm

[PyCharm Python Tutorial \(Simplilearn\)](#)

[Jet Brains Pycharm Website](#)

6.3.1. Links

[Pycharm Tutorial #1 - Setup & Basics](#)

[Pycharm Tutorial #2 - Debugging](#)

6.3.2. New Project

Start Project

File>New>Project

Enter in new window

In Project menu Right click project name

Enter file name

New>Python File name

Enter name

Add Library to Environment

File>Settings

Click on project name

Select: Project interpreter

Click '+' (upper right)

pick from list available libraries

click on install

wait for success message

6.3.3. Refactoring

6.3.4. Shortcuts

^F find in file

See recent changes

^D duplicate line cursor is on

todo list

can put on line where future change is needed

ctrl /

comment/uncomment selected lines

6.4. Visual Studio

[Microsoft Docs for Python in Visual Studio](#)

6.5. Komodo Edit

<https://www.activestate.com/komodo-edit>

7. Syntax

7.1. Reserved Words

<code>and</code>	<code>del</code>	<code>from</code>	<code>None</code>	<code>True</code>
<code>as</code>	<code>elif</code>	<code>global</code>	<code>nonlocal</code>	<code>try</code>
<code>assert</code>	<code>else</code>	<code>if</code>	<code>not</code>	<code>while</code>
<code>break</code>	<code>except</code>	<code>import</code>	<code>or</code>	<code>with</code>
<code>class</code>	<code>False</code>	<code>in</code>	<code>pass</code>	<code>yield</code>
<code>continue</code>	<code>finally</code>	<code>is</code>	<code>raise</code>	<code>async</code>
<code>def</code>	<code>for</code>	<code>lambda</code>	<code>return</code>	<code>await</code>

7.2. Import options

```
import somemodule
from somemodule import somefunction
from somemodule import somefunction, another function
```

```
import math as foobar
foobar.sqrt(4)
>2
```

```
from math import sqrt as foobar
foobar(4)
>2
```

8. Datatypes

8.1. Numerical Types

Two main **numerical types**

int

float

Other numerical types

Decimal

Fraction

Complex numbers (j or J)

Math operation: + - * /

/ always returns a float

// will return int discarding fractions

% will return remainder

** will return power, e.g. 5**2 = 25

= assigns a value to a variable

_ hold value of last printed expression

8.1.1. Math Functions

Casting

num = '100'

num = **int**(num)

Absolute

num = -3

num = **abs**(num)

Rounding

print (round(3.75)) result = 4

print (round(3.75, 1) result = 3.8

8.1.1.1. Math module

import math

for complete list of dot methods

search google: python 3 math module

8.2. String Types

[Python String Documentation](#)

Strings

'

" Strings can be enclosed with single or double quotes

\ String escape e.g. 'isn\' '

3 quotes can make a multi-line block (double quotes by convention)
+ Concatenate two strings

word[2] = character in position 2 e.g. r
word[-1] = character in position counting from end
word[2:5] characters from position 2 to 5
word[:2] # characters from start to pos 2
word[2:] # characters from pos 2 until end

Python strings can not be changed,
Strings are immutable
they can't be altered

Operations
index = 0
while index < len(fruit) :
 letter = fruit[index]
 index += 1

8.2.1. String formatting

There are three ways to perform string formatting.

1. The oldest method involves placeholders using the modulo % character.
2. An improved technique uses the .format() string method.
3. The newest method, introduced with Python 3.6, uses formatted string literals, called f-strings.

```
name = "John"  
age = 23
```

Modulo formatting

```
print('%s is %d years old' % (name, age))  
returns:  
John is 23 years old
```

```
or 'Hi %s I have %d donuts' % ('Alice', 42)
```

New Style

```
print(f'{name} is {age} years old')
```

.format method

```
print('{} is {} years old'.format(name, age))  
print('{} is {} years old'.format("John", 23))
```

f string formatting

```
print(f'{name} is {age} years old')
```

=====

Padding and Precision of Floating Point Numbers

Floating point numbers use the format %5.2f. Here, 5 would be the minimum number of characters the string should contain; these may be padded with leading whitespace if the entire number does not have this many digits. Next to this, .2f stands for how many numbers to show past the decimal point.

8.2.2. Methods

[String Methods](#)

String Methods

Strings implement all of the [common](#) sequence operations, along with the additional methods described below.

Strings also support two styles of string formatting, one providing a large degree of flexibility and customization (see `str.format()`, [Format String Syntax](#) and [Custom String Formatting](#)) and the other based on C `printf` style formatting that handles a narrower range of types and is slightly harder to use correctly, but is often faster for the cases it can handle ([printf-style String Formatting](#)).

The [Text Processing Services](#) section of the standard library covers a number of other modules that provide various text related utilities (including regular expression support in the `re` module).

Source: [String Methods](#) (18 August 2019 4:24 PM)

8.2.2.1. capitalize

- `str.capitalize()`
- Return a copy of the string with its first character capitalized and the rest lowercased.

8.2.2.2. casefold

`str.casefold()`

Return a casefolded copy of the string. Casefolded strings may be used for caseless matching.

Casefolding is similar to lowercasing but more aggressive because it is intended to remove all case distinctions in a string. For example, the German lowercase letter 'ß' is equivalent to "ss". Since it is already lowercase, `lower()` would do nothing to 'ß'; `casefold()` converts it to "ss".

The casefolding algorithm is described in section 3.13 of the Unicode Standard.

Source: [str.casefold\(\)](#) (18 August 2019 1:39 PM)

8.2.2.3. center

`str.center(width[, fillchar])`

Return centered in a string of length `width`. Padding is done using the specified `fillchar` (default is an ASCII space). The original string is returned if `width` is less than or equal to `len(s)`.

Source: [str.center\(width\[, fillchar\]\)](#) (18 August 2019 1:39 PM)

8.2.2.4. encode

`str.encode(encoding="utf-8", errors="strict")`

Return an encoded version of the string as a bytes object. Default encoding is 'utf-8'. `errors` may be given to set a different error handling scheme. The default for `errors` is 'strict', meaning that encoding errors raise a `UnicodeError`. Other possible values are 'ignore', 'replace',

'xmlcharrefreplace', 'backslashreplace' and any other name registered via `codecs.register_error()`, see section [Error Handlers](#). For a list of possible encodings, see section [Standard Encodings](#).

Source: [str.encode\(encoding="utf-8", errors="strict"\)](#) (18 August 2019 1:40 PM)

8.2.2.5. endswith

`str.endswith(suffix[, start[, end]])`

Return `True` if the string ends with the specified *suffix*, otherwise return `False`. *suffix* can also be a tuple of suffixes to look for. With optional *start*, test beginning at that position. With optional *end*, stop comparing at that position.

Source: [str.endswith\(suffix\[, start\[, end\]\]\)](#) (18 August 2019 1:41 PM)

8.2.2.6. expandtabs

`str.expandtabs(tabsize=8)`

Return a copy of the string where all tab characters are replaced by one or more spaces, depending on the current column and the given tab size. Tab positions occur every *tabsize* characters (default is 8, giving tab positions at columns 0, 8, 16 and so on). To expand the string, the current column is set to zero and the string is examined character by character. If the character is a tab (`\t`), one or more space characters are inserted in the result until the current column is equal to the next tab position. (The tab character itself is not copied.) If the character is a newline (`\n`) or return (`\r`), it is copied and the current column is reset to zero. Any other character is copied unchanged and the current column is incremented by one regardless of how the character is represented when printed.

```
>>> '01\t012\t0123\t01234'.expandtabs()
'01          012          0123          01234'
>>> '01\t012\t0123\t01234'.expandtabs(4)
'01      012      0123      01234'
```

Source: [str.expandtabs\(tabsize=8\)](#) (18 August 2019 1:41 PM)

8.2.2.7. find

`str.find(sub[, start[, end]])`

Return the lowest index in the string where substring *sub* is found within the slice `s[start:end]`. Optional arguments *start* and *end* are interpreted as in slice notation. Return `-1` if *sub* is not found.

Note

The `find()` method should be used only if you need to know the position of *sub*. To check if *sub* is a substring or not, use the `in` operator:

```
>>>
>>> 'Py' in 'Python'
True
```

8.2.2.8. format

`str.format(*args, **kwargs)`

Perform a string formatting operation. The string on which this method is called can contain literal text or replacement fields delimited by braces `{}`. Each replacement field contains either the numeric index of a positional argument, or the name of a keyword argument. Returns a copy of the string where each replacement field is replaced with the string value of the corresponding argument.

```
>>> "The sum of 1 + 2 is {}".format(1+2)
'The sum of 1 + 2 is 3'
```

See [Format String Syntax](#) for a description of the various formatting options that can be specified in format strings.

Note

When formatting a number (`int`, `float`, `complex`, `decimal.Decimal` and subclasses) with the `n` type (ex: `'{:n}'.format(1234)`), the function temporarily sets the `LC_CTYPE` locale to the `LC_NUMERIC` locale to decode `decimal_point` and `thousands_sep` fields of `localeconv()` if they are non-ASCII or longer than 1 byte, and the `LC_NUMERIC` locale is different than the `LC_CTYPE` locale. This temporary change affects other threads.

Source: [str.format\(*args, **kwargs\)](#) (18 August 2019 1:44 PM)

```
foo = "{},{},{ } are all letters"
print (foo.format('a', 'b', 'c'))
>a,b,c are all letters
```

```
foo = "{},{},{ } are all letters".format('a','b','c')
print(foo)
>a,b,c are all letters
```

```
# different datatypes can be used in formatting
print ("Hi ! My name is {} and I am {} years old"
      .format("User", 19))
```

```
# Keyword arguments are called
# by their keyword name
print("{gfg} is a {} science portal for {1}"
      .format("computer", "geeks", gfg ="GeeksforGeeks"))
```

```
# Python Program for
# Formatting of Strings
```

```
# Default order
String1 = "{} {} {}".format('Geeks', 'For', 'Life')
print("Print String in default order: ")
print(String1)
```

```
# Positional Formatting
String1 = "{1} {0} {2}".format('Geeks', 'For', 'Life')
print("\nPrint String in Positional order: ")
print(String1)
```

```
# Keyword Formatting
```

```
String1 = "{l} {f} {g}".format(g = 'Geeks', f = 'For', l = 'Life')
print("\nPrint String in order of Keywords: ")
print(String1)
```

8.2.2.9. format.map

`str.format_map(mapping)`

Similar to `str.format(**mapping)`, except that `mapping` is used directly and not copied to a `dict`. This is useful if for example `mapping` is a `dict` subclass:

```
>>> class Default(dict):
...     def __missing__(self, key):
...         return key
>>> '{name} was born in {country}'.format_map(Default(name='Guido'))
'Guido was born in country'
```

New *in* version 3.2.

Source: [str.format_map\(mapping\)](#) (18 August 2019 4:05 PM)

8.2.2.10. index

`str.index(sub[, start[, end]])`

Like `find()`, but raise `ValueError` when the substring is not found.

Source: [str.index\(sub\[, start\[, end\]\]\)](#) (18 August 2019 4:06 PM)

8.2.2.11. isalnum

`str.isalnum()`

Return true if all characters in the string are alphanumeric and there is at least one character, false otherwise. A character `c` is alphanumeric if one of the following returns `True`: `c.isalpha()`, `c.isdecimal()`, `c.isdigit()`, or `c.isnumeric()`.

Source: [str.isalnum\(\)](#) (18 August 2019 4:07 PM)

8.2.2.12. isalpha

`str.isalpha()`

Return true if all characters in the string are alphabetic and there is at least one character, false otherwise. Alphabetic characters are those characters defined in the Unicode character database as “Letter”, i.e., those with general category property being one of “Lm”, “Lt”, “Lu”, “LI”, or “Lo”. Note that this is different from the “Alphabetic” property defined in the Unicode Standard.

Source: [str.isalpha\(\)](#) (18 August 2019 4:07 PM)

8.2.2.13. isascii

`str.isascii()`

Return true if the string is empty or all characters in the string are ASCII, false otherwise. ASCII characters have code points in the range U+0000-U+007F.

New in version 3.7

Source: [str.isascii\(\)](#) (18 August 2019 4:08 PM)

8.2.2.14. isdecimal

`str.isdecimal()`

Return true if all characters in the string are decimal characters and there is at least one character, false otherwise. Decimal characters are those that can be used to form numbers in base 10, e.g. U+0660, ARABIC-INDIC DIGIT ZERO. Formally a decimal character is a character in the Unicode General Category "Nd".

Source: [str.isdecimal\(\)](#) (18 August 2019 4:08 PM)

8.2.2.15. isdigit

`str.isdigit()`

Return true if all characters in the string are digits and there is at least one character, false otherwise. Digits include decimal characters and digits that need special handling, such as the compatibility superscript digits. This covers digits which cannot be used to form numbers in base 10, like the Kharosthi numbers. Formally, a digit is a character that has the property value `Numeric_Type=Digit` or `Numeric_Type=Decimal`.

Source: [str.isdigit\(\)](#) (18 August 2019 4:10 PM)

8.2.2.16. isidentifier

`str.isidentifier()`

Return true if the string is a valid identifier according to the language definition, section [Identifiers and keywords](#).

Use `keyword.iskeyword()` to test for reserved identifiers such as `def` and `class`.

Source: [str.isidentifier\(\)](#) (18 August 2019 4:11 PM)

8.2.2.17. islower

`str.islower()`

Return true if all cased characters `4` in the string are lowercase and there is at least one cased character, false otherwise.

Source: [str.islower\(\)](#) (18 August 2019 4:11 PM)

8.2.2.18. isnumeric

`str.isnumeric()`

Return true if all characters in the string are numeric characters, and there is at least one character, false otherwise. Numeric characters include digit characters, and all characters that have the Unicode numeric value property, e.g. U+2155, VULGAR FRACTION ONE FIFTH. Formally, numeric characters are those with the property value `Numeric_Type=Digit`, `Numeric_Type=Decimal` or `Numeric_Type=Numeric`.

Source: [str.isnumeric\(\)](#) (18 August 2019 4:11 PM)

8.2.2.19. isprintable

`str.isprintable()`

Return true if all characters in the string are printable or the string is empty, false otherwise. Nonprintable characters are those characters defined in the Unicode character database as "Other" or "Separator", excepting the ASCII space (0x20) which is considered printable. (Note that printable characters in this context are those which should not be escaped when `repr()` is invoked on a string. It has no bearing on the handling of strings written to `sys.stdout` or `sys.stderr`.)

Source: [str.isprintable\(\)](#) (18 August 2019 4:13 PM)

8.2.2.20. isspace

`str.isspace()`

Return true if there are only whitespace characters in the string and there is at least one character, false otherwise. Whitespace characters are those characters defined in the Unicode character database as "Other" or "Separator" and those with bidirectional property being one of "WS", "B", or "S".

Source: [str.isspace\(\)](#) (18 August 2019 4:13 PM)

8.2.2.21. istitle

`str.istitle()`

Return true if the string is a titlecased string and there is at least one character, for example uppercase characters may only follow uncased characters and lowercase characters only cased ones. Return false otherwise.

Source: [str.isitle\(\)](#) (18 August 2019 4:14 PM)

8.2.2.22. isupper

`str.isupper()`

Return true if all cased characters `4` in the string are uppercase and there is at least one cased character, false otherwise.

Source: [str.isupper\(\)](#) (18 August 2019 4:14 PM)

8.2.2.23. join

`str.join(iterable)`

Return a string which is the concatenation of the strings in *iterable*. A `TypeError` will be raised if there are any non-string values in *iterable*, including `bytes` objects. The separator between elements is the string providing this method.

Source: [str.join\(iterable\)](#) (18 August 2019 4:16 PM)

```
seq=['1','2','3','4','5','6']
s='- '
s = s.join(seq)
print s
print(s)
1-2-3-4-5-6
>>>
```

8.2.2.24. ljust

`str.ljust(width[, fillchar])`

Return the string left justified in a string of length *width*. Padding is done using the specified *fillchar* (default is an ASCII space). The original string is returned if *width* is less than or equal to `len(s)`.

Source: [str.ljust\(width\[, fillchar\]\)](#) (18 August 2019 4:16 PM)

8.2.2.25. lower

`str.lower()`

Return a copy of the string with all the cased characters `4` converted to lowercase. The lowercasing algorithm used is described in section 3.13 of the Unicode Standard.

Source: [str.lower\(\)](#) (18 August 2019 4:16 PM)

8.2.2.26. lstrip

`str.lstrip([chars])`

Return a copy of the string with leading characters removed. The *chars* argument is a string specifying the set of characters to be removed. If omitted or `None`, the *chars* argument defaults to removing whitespace. The *chars* argument is not a prefix; rather, all combinations of its values are stripped:

```
>>>
>>> 'spacious'.lstrip()
'spacious'
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

Source: [str.lstrip\(\[chars\]\)](#) (18 August 2019 4:18 PM)

8.2.2.27. maketrans

`static str.maketrans(x[, y[, z]])`

This static method returns a translation table usable for `str.translate()`.

If there is only one argument, it must be a dictionary mapping Unicode ordinals (integers) or characters (strings of length 1) to Unicode ordinals, strings (of arbitrary lengths) or `None`. Character keys will then be converted to ordinals.

If there are two arguments, they must be strings of equal length, and in the resulting dictionary, each character in *x* will be mapped to the character at the same position in *y*. If there is a third argument, it must be a string, whose characters will be mapped to `None` in the result.

Source: [static str.maketrans\(x\[, y\[, z\]\]\)](#) (18 August 2019 4:20 PM)

8.2.2.28. partition

`str.partition(sep)`

Split the string at the first occurrence of *sep*, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing the string itself, followed by two empty strings.

Source: [str.partition\(sep\)](#) (18 August 2019 4:20 PM)

8.2.2.29. replace

`str.replace(old, new[, count])`

Return a copy of the string with all occurrences of substring *old* replaced by *new*. If the optional argument *count* is given, only the first *count* occurrences are replaced.

Source: [str.replace\(old, new\[, count\]\)](#) (18 August 2019 4:21 PM)

8.2.2.30. rfind

`str.rfind(sub[, start[, end]])`

Return the highest index in the string where substring *sub* is found, such that *sub* is contained within `s[start:end]`. Optional arguments *start* and *end* are interpreted as in slice notation. Return `-1` on failure.

Source: [str.rfind\(sub\[, start\[, end\]\]\)](#) (18 August 2019 4:21 PM)

8.2.2.31. rindex

`str.rindex(sub[, start[, end]])`

Like `rfind()` but raises `ValueError` when the substring *sub* is not found.

Source: [str.rindex\(sub\[, start\[, end\]\]\)](#) (18 August 2019 4:21 PM)

8.2.2.32. rjust

`str.rjust(width[, fillchar])`

Return the string right justified in a string of length *width*. Padding is done using the specified *fillchar* (default is an ASCII space). The original string is returned if *width* is less than or equal to `len(s)`.

Source: [str.rjust\(width\[, fillchar\]\)](#) (18 August 2019 4:22 PM)

8.2.2.33. rpartition

`str.rpartition(sep)`

Split the string at the last occurrence of *sep*, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing two empty strings, followed by the string itself.

Source: [str.rpartition\(sep\)](#) (18 August 2019 4:22 PM)

8.2.2.34. rsplit

`str.rsplit(sep=None, maxsplit=-1)`

Return a list of the words in the string, using *sep* as the delimiter string. If *maxsplit* is given, at most *maxsplit* splits are done, the *rightmost* ones. If *sep* is not specified or `None`, any whitespace string is a separator. Except for splitting from the right, `rsplit()` behaves like `split()` which is described in detail below.

Source: [str.rsplit\(sep=None, maxsplit=-1\)](#) (18 August 2019 4:25 PM)

8.2.2.35.rstrip

`str.rstrip([chars])`

Return a copy of the string with trailing characters removed. The *chars* argument is a string specifying the set of characters to be removed. If omitted or `None`, the *chars* argument defaults to removing whitespace. The *chars* argument is not a suffix; rather, all combinations of its values are stripped:

```
>>>
>>> 'spacious'.rstrip()
'spacious'
>>> 'mississippi'.rstrip('ipz')
'mississ'
```

Source: [str.rstrip\(\[chars\]\)](#) (18 August 2019 4:26 PM)

8.2.2.36.split

`str.split(sep=None, maxsplit=-1)`

Return a list of the words in the string, using *sep* as the delimiter string. If *maxsplit* is given, at most *maxsplit* splits are done (thus, the list will have at most *maxsplit*+1 elements). If *maxsplit* is not specified or `-1`, then there is no limit on the number of splits (all possible splits are made).

If *sep* is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (for example, `'1,,2'.split(',')` returns `['1', '', '2']`). The *sep* argument may consist of multiple characters (for example, `'1<>2<>3'.split('<>')` returns `['1', '2', '3']`). Splitting an empty string with a specified separator returns `['']`.

For example:

```
>>>
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3'.split(',')
['1', '2', '', '3']
```

If *sep* is not specified or is `None`, a different splitting algorithm is applied: runs of consecutive whitespace are regarded as a single separator, and the result will contain no empty strings at the start or end if the string has leading or trailing whitespace. Consequently, splitting an empty string or a string consisting of just whitespace with a `None` separator returns `[]`.

For example:

```
>>>
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> '1 2 3'.split(' ')
['1', '2', '3']
```

Source: [str.split\(sep=None, maxsplit=-1\)](#) (18 August 2019 4:26 PM)

8.2.2.37.splitlines

`str.splitlines([keepends])`

Return a list of the lines in the string, breaking at line boundaries. Line breaks are not included in the resulting list unless *keepends* is given and true. This method splits on the following line boundaries. In particular, the boundaries are a superset of [universal newlines](#).

Representation	Description
<code>\n</code>	Line Feed
<code>\r</code>	Carriage Return
<code>\r\n</code>	Carriage Return + Line Feed
<code>\v</code> or <code>\x0b</code>	Line Tabulation
<code>\f</code> or <code>\x0c</code>	Form Feed
<code>\x1c</code>	File Separator
<code>\x1d</code>	Group Separator
<code>\x1e</code>	Record Separator
<code>\x85</code>	Next Line (C1 Control Code)
<code>\u2028</code>	Line Separator
<code>\u2029</code>	Paragraph Separator

Changed in version 3.2: `\v` and `\f` added to list of line boundaries. For example:

```
>>>
>>> 'ab\nde fg\rkl\r\n'.splitlines()
['ab', 'de', 'fg', 'kl']
>>> 'ab\nde fg\rkl\r\n'.splitlines(keepends=True)
['ab\nde', 'fg\r', 'kl\r\n']
```

Unlike `split()` when a delimiter string *sep* is given, this method returns an empty list for the empty string, and a terminal line break does not result in an extra line:

```
>>>
>>> "".splitlines()
[]
>>> "One\n".splitlines()
['One']
```

For comparison, `split('\n')` gives:

```
>>>
>>> ''.split('\n')
['']
>>> 'Two\n'.split('\n')
['Two', '']
```

Source: [str.splitlines\(\[keepends\]\)](#) (18 August 2019 4:28 PM)

8.2.2.38. startswith

`str.startswith(prefix[, start[, end]])`

Return `True` if string starts with the *prefix*, otherwise return `False`. *prefix* can also be a tuple of prefixes to look for. With optional *start*, test string beginning at that position. With optional *end*, stop comparing string at that position.

Source: [str.startswith\(prefix\[, start\[, end\]\]\)](#) (18 August 2019 4:29 PM)

8.2.2.39. strip

`str.strip([chars])`

Return a copy of the string with the leading and trailing characters removed. The *chars* argument is a string specifying the set of characters to be removed. If omitted or `None`, the *chars* argument defaults to removing whitespace. The *chars* argument is not a prefix or suffix; rather, all combinations of its values are stripped:

```
>>>
>>> 'spacious'.strip()
'spacious'
>>> 'www.example.com'.strip('cmowz.')
'example'
```

The outermost leading and trailing *chars* argument values are stripped from the string. Characters are removed from the leading end until reaching a string character that is not contained in the set of characters in *chars*. A similar action takes place on the trailing end. For example:

```
>>>
>>> comment_string = '#..... Section 3.2.1 Issue #32 .....'
>>> comment_string.strip('#!')
'Section 3.2.1 Issue #32'
```

Source: [str.strip\(\[chars\]\)](#) (18 August 2019 4:29 PM)

8.2.2.40. swapcase

`str.swapcase()`

Return a copy of the string with uppercase characters converted to lowercase and vice versa. Note that it is not necessarily true that `s.swapcase().swapcase() == s`.

Source: [str.swapcase\(\)](#) (18 August 2019 4:30 PM)

8.2.2.41. title

`str.title()`

Return a titlecased version of the string where words start with an uppercase character and the remaining characters are lowercase. For example:

```
>>>
>>> 'Hello world'.title()
'Hello World'
```

The algorithm uses a simple language-independent definition of a word as groups of consecutive letters. The definition works in many contexts but it means that apostrophes in contractions and possessives form word boundaries, which may not be the desired result:

```
>>>
>>> "they're bill's friends from the UK".title()
'They'Re Bill'S Friends From The Uk'
```

A workaround for apostrophes can be constructed using regular expressions:

```
>>>
>>> import re
>>> def titlecase(s):
...     return re.sub(r"[A-Za-z]+('[A-Za-z]+)?",
...                    lambda mo: mo.group(0)[0].upper() +
...                               mo.group(0)[1:].lower(),
...                    s)
>>> titlecase("they're bill's friends.")
'They're Bill's Friends.'
```

Source: [str.title\(\)](#) (18 August 2019 4:30 PM)

8.2.2.42. translate

`str.translate(table)`

Return a copy of the string in which each character has been mapped through the given translation table. The table must be an object that implements indexing via `__getitem__()`, typically a [mapping](#) or [sequence](#). When indexed by a Unicode ordinal (an integer), the table object can do any of the following: return a Unicode ordinal or a string, to map the character to one or more other characters; return `None`, to delete the character from the return string; or raise a [LookupError](#) exception, to map the character to itself.

You can use `str.maketrans()` to create a translation map from character-to-character mappings in different formats.

See also the `codecs` module for a more flexible approach to custom character mappings.

Source: [str.translate\(table\)](#) (18 August 2019 4:32 PM)

8.2.2.43. upper

`str.upper()`

Return a copy of the string with all the cased characters [4](#) converted to uppercase. Note that `s.upper().isupper()` might be `False` if `s` contains uncased characters or if the Unicode category of the resulting character(s) is not “Lu” (Letter, uppercase), but e.g. “Lt” (Letter, titlecase). The uppercasing algorithm used is described in section 3.13 of the Unicode Standard.

Source: [str.upper\(\)](#) (18 August 2019 4:32 PM)

8.2.2.44. zfill

`str.zfill(width)`

Return a copy of the string left filled with ASCII '0' digits to make a string of length *width*. A leading sign prefix ('+'/'-') is handled by inserting the padding *after* the sign character rather than before. The original string is returned if *width* is less than or equal to `len(s)`.

For

example:

```
>>>
>>> "42".zfill(5)
'00042'
>>> "-42".zfill(5)
'-0042'
```

Source: [str.zfill\(width\)](#) (18 August 2019 4:32 PM)

8.3. Bool

Bool values are True and False

Any numerical value other than 0 is false

Any empty string is False

8.4. Collection Types

8.4.1. Lists

Lists set off by square brackets with comma separators

The items can be any type, such as ints, strings, or other lists or a mixture of types.

operations

```
letters = ['a', 'b', 'c']
```

```
numbers = [1,2,3]
```

```
print([list])
```

```
for i in list:
```

```
    print(i)
```

```
for i in range(len(list)):
```

Concatenating lists using + -> lists a+b

```
line.split(',')
```

value in list

```
'b' in letters
```

Access of individual items

`letters[1]` provides 'b'

slicing works the same as in strings

`numbers[::2]` will access every other item

list unpacking

assignment of list item to an external variable,e.g.
numbers[1,2,3]
first = numbers[0]

list copies

b = a Point to same area in memory
b = a[:] Makes identical copy

methods

list.append() Add Single Element to The List
list.extend() Add Elements of a List to Another List
list.insert() Inserts Element to The List
list.remove() Removes Element by name from the List
list.index() returns smallest index of element in list
list.count() returns occurrences of element in a list
list.pop() Removes Element at Given Index, last by default
list.reverse() Reverses a List
list.sort() sorts elements of a list
list.copy() Returns Shallow Copy of a List
list.clear() Removes all Items from the List

built-in functions that can take list as a parameter and perform some task

any() Checks if any Element of an Iterable is True
all() returns true when all elements in iterable is true
ascii() Returns String Containing Printable Representation
bool() Converts a Value to Boolean
enumerate() Returns an Enumerate Object
filter() constructs iterator from elements which are true
iter() returns iterator for an object
list() Function creates list in Python
len() Returns Length of an Object
max() returns largest element
min() returns smallest element
map() Applies Function and Returns a List
reversed() returns reversed iterator of a sequence
slice() creates a slice object specified by range()
sorted() returns sorted list from a given iterable
sum() Add items of an Iterable
zip() Returns an Iterator of Tuples

Custom sorts using key

```
a = ['ccc', 'aaaz', 'd', 'bb']  
sorted(a, key=len)  
where key can be any function
```

8.4.1.1. Strings

To convert from a string to a list of characters, you can use list:

```
>>> s = 'spam'  
>>> t = list(s)  
>>> print(t)  
['s', 'p', 'a', 'm']
```

If you want to break a
string into words, you can use the split method:

```
>>> s = 'pining for the fjords'  
>>> t = s.split()  
>>> print(t)  
['pining', 'for', 'the', 'fjords']  
>>> print(t[2])
```

8.4.2. Tuples

Tuples are similar to lists except that you can't modify tuples and is said to be immutable. Therefore a tuple is a **read-only list**

Rather than square brackets tuples uses **parens**

```
months = ('Jan', 'Feb', 'Mar', 'Apr', 'Jun')
```

A tuple has the same functions as a list except for those functions that mutates the list, but there are only two methods, i.e. `tuple.count` and `tuple.index`

`months.index('Feb')` returns the index, i.e. 1

`months.count('Feb')` returns the numbers of occurrences, i.e. 1

If the argument is a sequence (string, list, or tuple), the result of the call to `tuple` is a tuple with the elements of the sequence:

```
>>> t = tuple('lupins')
>>> print(t)
('l', 'u', 'p', 'i', 'n', 's')
```

A list can be converted to a tuple

```
list = tuple(numbers)
```

8.4.3. Sets

sets are values that are unordered and have no duplicates

differ in that they are set off with curly braces

```
months = {'Jan', 'Feb', 'Mar', 'Apr', 'Jun'}
```

is often used to get rid of duplicates

is often used to set what are differences or identicals between lists

!	Union	Combines two sets
&	Intersection	Gets items only in both
-	Difference	Gets items in first but not second set
^	Symmetric Difference	Gets items in either set but not both

```
first = {1, 2, 3, 4, 5, 6}
second = {4, 5, 6, 7, 8, 9}
```

```
print(first | second)
print(first & second)
print(first - second)
print(second - first)
print(first ^ second)
```

Result:

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
{4, 5, 6}
```

```
{1, 2, 3}
{8, 9, 7}
{1, 2, 3, 7, 8, 9}
```

8.4.3.1. Methods

A collection with no duplicates, Items are unordered.

If you want to remove duplicates from list, convert to set.

```
numbers = [1,2,2,3,4]
uniques = set(numbers)
```

Method	Description
Python List append()	Add Single Element to The List
Python List extend()	Add Elements of a List to Another List
Python List insert()	Inserts Element to The List
Python List remove()	Removes Element from the List
Python List index()	returns smallest index of element in list
Python List count()	returns occurrences of element in a list
Python List pop()	Removes Element at Given Index
Python List reverse()	Reverses a List
Python List sort()	sorts elements of a list
Python List copy()	Returns Shallow Copy of a List
Python List clear()	Removes all Items from the List

Python any()	Checks if any Element of an Iterable is True
Python all()	returns true when all elements are true
Python ascii()	Printable Representation
Python bool()	Converts a Value to Boolean
Python enumerate()	Returns an Enumerate Object
Python filter()	constructs iterator from true elements
Python iter()	returns iterator for an object
Python list() Function	creates list in Python
Python len()	Returns Length of an Object
Python max()	returns largest element
Python min()	returns smallest element
Python map()	Applies Function and Returns a List
Python reversed()	returns reversed iterator of a sequence
Python slice()	creates a slice object specified by range()
Python sorted()	returns sorted list from a given iterable
Python sum()	Add items of an Iterable
Python zip()	Returns an Iterator of Tuples

8.4.4. Dictionaries

A dictionary is a collection of key, value pairs which are set off with curly braces {}

The key and value pairs are separated with colon and the pairs separated by commas

The ordering of the pairs is not constant from time to time and is unpredictable.

```
point = {"x":1, "y":2}
point = dict(x=1, y=2)
```

Adding pairs

```
d = dict()
```

```
print(d)
{}
d['one'] = 'uno'
print(d)
{'one': 'uno'}
```

Inclusion

The in operator works in dictionaries on keys but not values. To find if a value is included use function dict.values() e.g.
 vals = list(d.values())
 target in vals

Sorting

To sort a dictionary, you must make a list of keys and sort that list, e.g.
 lst = list(d.keys())
 lst.sort()
 for key in lst:
 print(key, d)

dict.clear()	Removes all Items
dict.copy()	Returns Shallow Copy of a Dictionary
dict.fromkeys()	creates dictionary from given sinnequence
dict.get()	Returns Value of The Key
dict.items()	returns view of dictionary's (key, value) pair
dict.keys()	Returns View Object of All Keys
dict.popitem()	Returns & Removes Element From Dictionary
dict.setdefault()	Inserts Key With a Value if Key is not Present
dict.pop()	removes and returns element having given key
dict.values()	returns view of all values in dictionary
dict.update()	Updates the Dictionary

any()	Checks if any Element of an Iterable is True
all()	returns true when all elements in iterable is true
ascii()	Returns String Containing Printable Representation
bool()	Converts a Value to Boolean
dict()	Creates a Dictionary
enumerate()	Returns an Enumerate Object
filter()	constructs iterator from elements which are true
iter()	returns iterator for an object
len()	Returns Length of an Object
max()	returns smallest element
map()	Applies Function and Returns a List
sorted()	returns sorted list from a given iterable
sum()	Add items of an Iterable
zip()	Returns an Iterator of Tuples

8.5. type converstion

```
int(x)
float(x)
bool(x)
str(x)
```

```
type(x)
```

9. Flow Control

9.1. if

Perhaps the most well-known statement type is the `if` statement. If statements must be followed with a colon.

```
>>>
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
More
```

There can be zero or more `elif` parts, and the `else` part is optional. The keyword 'elif' is short for 'else if', and is useful to avoid excessive indentation. An `if ... elif ... elif ...` sequence is a substitute for the `switch` or `case` statements found in other languages.

Source: [4. More Control Flow Tools — Python 3.7.4 documentation](#) (13 August 2019 1:29 PM)

9.2. for loop

Python's `for` statement iterates over the items of any sequence (a list or a string), in the order that they appear in the sequence.

```
nums = [1,2,3,4,5]
```

```
for num in nums:
    print(num)
```

`break` exits loop entirely
`continue` returns to loop start

Range statement options

```
# Prints 1,2,3,4,5
for x in range(5):
    print(x)
```

```
# Prints out 3,4,5
for x in range(3, 6):
    print(x)
```

```
# Prints out 3,5,7
for x in range(3, 8, 2):
    print(x)
```

9.3. while loop

While loops repeat as long as a certain boolean condition is met. For example:

```
# Prints out 0,1,2,3,4
count = 0
```

```
while count < 5:
    print(count)
    count += 1
```

9.4. range function

The range function will iterate over a sequence of numbers,e.g.

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

range(5)	Iterate over 5 numbers
range(5,10)	Process to range 10 start at 5
range(0,10,3)	Every 3rd item
range(0,10,-2)	Every other item starting at end

9.5. Loop Controls

Break statement

The `break` statement, like in C, breaks out of the innermost enclosing `for` or `while` loop. Loop statements may have an `else` clause; it is executed when the loop terminates through exhaustion of the list (with `for`) or when the condition becomes false (with `while`), but not when the loop is terminated by a `break` statement

Continue statement

The `continue` statement, also borrowed from C, continues with the next iteration of the loop:

Else

Executes only if loop ends normally. Will not executed if loop ends with a `break` statement.

9.6. pass

The `pass` statement does nothing. It can be used when a statement is required syntactically but the program requires no action.

It is sometimes used as a placeholder for code that will be written later.

10. Operators

10.1. Arithmetic

Addition	3 + 2
Subtraction	3 - 2
Multiplication	3 * 2
Division	3 / 2
Integer Division	3 // 2
Modulus	3 % 2
Exponent	3 ** 2

augment assignment

num += 1 adds 1
num *= 10 multiplies by ten

10.2. Bitwise Operators

&	And
!	Or
^	Xor
<<	Shift left
>>	Shift right

10.3. Comparison

Comparison

Equal	3 == 2
Not equal:	3 != 2
Greater than:	3 > 2
Less than	3 < 2
Greater or Equal	3 >= 2
Less or equal:	3 <= 2

x is y x and y are the same object
x is not y
x in y x is a member of the container y
x not in y
Can also use these symbols to compare strings
"bag" > "apple" true
"bag" == "BAG" false

10.4. Boolean Operators

Logical operators

and	And
or	Or

not	Not - reverses a boolean
in	Value in set
not in	Value not in set
is	Same object identity
is not	Not same object identity

10.5. Precedence

Operator Precedence

Operator	Description
**	Exponent
+X, -X	Positive, negative
*, /, //, %	Multiplication, division, remainder
+, -	Addition and subtraction
<<, >>	Bitwise shifts
&	Bitwise AND
^	Bitwise XOR
 	Bitwise OR
in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, including membership tests and identity tests
not x	Boolean NOT
and	Boolean AND
or	Boolean OR

PEMDAS

Complete chart:
docs.python.org/3/reference/expressions.html

11. Howto

11.1. Sorting

sorted()

last two parameters optional
sorted(iterable, key, reverse=False)
iterable can be list,tuple,string,dictionary, or set
returns a sorted list (new)

sort()

all parameters optional
values.sort(key, reverse=false)
sorts original list, returns nothing

e.g

```
numbers = [1,3,4,5]
numbers.sort()
```

Using key

sorted() function has an optional parameter called 'key' which takes a function as its value. This key function transforms each element before sorting, it takes the value and returns 1 value which is then used within sort instead of the original value.

e.g. sorting on length

```
L = ['cccc', 'b', 'dd','aaa']
sorted(L, key = len)
```

key() also takes a user-defined function for basis of sorting

```
L = [15,3,11,7]
```

```
def func(x):
    return x % 7
```

```
result = sorted(L, key = func)
result = [7,15,3,11]
```

11.2. Range

Parameters: start, stop, stepsize

```
for num in range(10):
    generates nums 0-9
```

```
for num in range(3,10):
    generates nums 3-9
```

```
for num in range(0,10,2):
    generates even nums 0-8
```

It doesn't return anything, for return
R=list(range(0,11,2))

11.3. Enumerate

Enumerate is a built-in function of Python. It allows us to loop over something and have an automatic counter.

```
word = 'abcde'
for item in enumerate(word):
    print(item)
```

returns

```
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
(4, 'e')
```

==

```
for index, letter in enumerate(word)
    print(index)
    print(letter)
```

=====

```
my_list = ['apple', 'banana', 'grapes', 'pear']
for c, value in enumerate(my_list, 1):
    print(c, value)
# Output:
# 1 apple
# 2 banana
# 3 grapes
# 4 pear
```

Source: [13. Enumerate](#) (3 December 2020 12:07 PM)

```
my_list = ['apple', 'banana', 'grapes', 'pear']
counter_list = list(enumerate(my_list, 1))
print(counter_list)
# Output: [(1, 'apple'), (2, 'banana'), (3, 'grapes'), (4, 'pear')]
```

Source: [13. Enumerate](#) (3 December 2020 12:07 PM)

11.4. Zip

Generator function that combines two lists

```
mylist1 = [1,2,3,4,5]
mylist2 = ['a','b','c','d','e']

list(zip(mylist1,mylist2))
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
=====
```

```
for item1, item2 in zip(mylist1,mylist2):
    print(item1,item2)
1 a
2 b
3 c
4 d
```

11.5. Input

Takes input from user

```
name = input("Enter your name: ")
print("Hello", name)
```

Input always returns string, often it must be cast to something else.

11.6. *args

*args permits the passing of any number of parameters to a function, e.g.

```
def myfunc(*args):
    return sum(args)
```

```
result = myfunc(20,30, 70)
print(result)
```

****kwargs**

Applies to dictionaries

11.7. map

A map is used to call the same function for every item in a list of other iterable.

```
def square (num):
    return num**2
```

```
num = [1,2,3,4,5]
for item in map(square, num):
    print(item)
or
list(map(square, num))
```

11.8. filter

A filter converts a list into a list containing only the items that return True

```
def check_even(num):
    return num % 2 == 0
```

```
nums = [1,2,3,4,5,6,7,8,9,10]
filter(check_even, nums)
```

12. Functions

There are two types of functions:

1. Perform a task
2. Return a value

The following statement is common. It indicates that main should be executed from this file as opposed to some file that has been imported.

```
if __name__ == '__main__': main()
```

It also causes the file to be completely read before being executed. This means a function can be called which is located later in the file.

12.1. Definition

function definition example

Function starts with def (define), followed by function name, parentheses(), and colon(:). Everything which is indented below a function definition is a part of that function. The function is not executed until it is called.

Here 'message' is a global variable. It could also be a local variable.

```
message = "Hello World"
def main():
    print(message)

main()
```

12.2. Arguments

Function can include arguments

```
def main():
    myfunction(5,6,7)

def myfunction(a,b,c):
    print(a,b,c)
```

An error results if called function doesn't receive all arguments.

Changing a value in a called function will not change its value in the caller. An exception is mutable values such as in list.

default arguments

```
def main():
    myfunction(5,6,7)

def myfunction(a,b,c=9):
    print(a,b,c)
```

A default argument can be provided. In that case, the caller no longer is required to provide a value for the argument. If it does, it overrides the default value.

All default values must be provided after the non-defaults.

optional number of arguments

Python allows an optional number of arguments which is treated as a tuple.

```
def add(*args):
    total=0
    for n in args:
        total += n
    return total
```

```
print(add(2,3,4,5))
```

Here the '*' in parameter means a tuple or set of values
It is traditional to call the tuple argument *args

A function can also return multiple values:

```
return(x1*x2), (x1/x2)
mult, divide = mult_div(5,4)
```

Source: [__return\(x1*x2\), \(x1/x2\)](#) (17 October 2019 2:13 PM)

Keyword arguments

dictionaries can be passed with syntax
def main(**kwarg):

12.3. Return

All functions return a value whether specified or not. Unlike some languages, a function and a procedure.

Python functions are capable of returning simple or complex values.

```
def main():
    x = func()
    print(type(x),x)
```

```
def func():
    return dict(x=42, y=43, z=44)
```

```
if __name__ == '__main__': main()
```

function returns:

```
<class 'dict'> {'x': 42, 'y': 43, 'z': 44}
=====
```

```
def student_info(*args, **kwargs):
    print(args)
    print(kwargs)
```

```
student_info('Math', 'Art', name = 'John', age = '22')
```

Returns:
('Math', 'Art')

```
{'name': 'John', 'age': '22'}
```

12.4. Scope

global variables are applied to whole file
local variables are applied only within function

values of global variables can not be changed in a function unless it is defined there as global

```
message = 'a'
```

```
def greet(name)
    global message = 'b'
```

12.5. Built-in Functions

[Built in Functions - Python Doc](#)

-
- | Function | Description |
|----------------------------|---|
| <code>abs()</code> | Returns the absolute value of a number |
| <code>all()</code> | Returns True if all items in an iterable object are true |
| <code>any()</code> | Returns True if any item in an iterable object is true |
| <code>ascii()</code> | Returns a readable version of an object. Replaces none-ascii characters with escape character |
| <code>bin()</code> | Returns the binary version of a number |
| <code>bool()</code> | Returns the boolean value of the specified object |
| <code>bytearray()</code> | Returns an array of bytes |
| <code>bytes()</code> | Returns a bytes object |
| <code>callable()</code> | Returns True if the specified object is callable, otherwise False |
| <code>chr()</code> | Returns a character from the specified Unicode code. |
| <code>classmethod()</code> | Converts a method into a class method |
| <code>compile()</code> | Returns the specified source as an object, ready to be executed |
| <code>complex()</code> | Returns a complex number |
| <code>delattr()</code> | Deletes the specified attribute (property or method) from the specified object |
| <code>dict()</code> | Returns a dictionary (Array) |
| <code>dir()</code> | Returns a list of the specified object's properties and methods |
| <code>divmod()</code> | Returns the quotient and the remainder when argument1 is divided by argument2 |
| <code>enumerate()</code> | Takes a collection (e.g. a tuple) and returns it as an enumerate object |

- | | | |
|----------------|--|----------|
| • eval() | Evaluates and executes an expression | |
| • exec() | Executes the specified code (or object) | |
| • filter() | Use a filter function to exclude items in an object | iterable |
| • float() | Returns a floating point number | |
| • format() | Formats a specified value | |
| • frozenset() | Returns a frozenset object | |
| • getattr() | Returns the value of the specified attribute (property or method) | |
| • globals() | Returns the current global symbol table as a dictionary | |
| • hasattr() | Returns True if the specified object has the specified attribute (property/method) | |
| • hash() | Returns the hash value of a specified object | |
| • help() | Executes the built-in help system | |
| • hex() | Converts a number into a hexadecimal value | |
| • id() | Returns the id of an object | |
| • input() | Allowing user input | |
| • int() | Returns an integer number | |
| • isinstance() | Returns True if a specified object is an instance of a specified object | |
| • isinstance() | Returns True if a specified class is a subclass of a specified object | |
| • iter() | Returns an iterator object | |
| • len() | Returns the length of an object | |
| • list() | Returns a list | |
| • locals() | Returns an updated dictionary of the current local symbol table | symbol |
| • map() | Returns the specified iterator with the specified function applied to each item | |
| • max() | Returns the largest item in an iterable | |
| • memoryview() | Returns a memory view object | |
| • min() | Returns the smallest item in an iterable | |
| • next() | Returns the next item in an iterable | |
| • object() | Returns a new object | |
| • oct() | Converts a number into an octal | |
| • open() | Opens a file and returns a file object | |
| • ord() | Convert an integer representing the Unicode of the specified character | |
| • pow() | Returns the value of x to the power of y | |
| • print() | Prints to the standard output device | |
| • property() | Gets, sets, deletes a property | |
| • range() | Returns a sequence of numbers, starting from 0 and increments by 1 (by default) | |
| • repr() | Returns a readable version of an object | |
| • reversed() | Returns a reversed iterator | |
| • round() | Rounds a numbers | |
| • set() | Returns a new set object | |

- `setattr()` Sets an attribute (property/method) of an object
- `slice()` Returns a slice object
- `sorted()` Returns a sorted list
- `@staticmethod()` Converts a method into a static method
- `str()` Returns a string object
- `sum()` Sums the items of an iterator
- `tuple()` Returns a tuple
- `type()` Returns the type of an object
- `vars()` Returns the `__dict__` property of an object
-

12.6. Lambda

```
g = lambda x: 3*x+1
g(2)
returns 7
```

multi expression:

```
lambda x,y,z: single line statement
=====
```

filters

```
nums = [11, 22, 33, 44, 55]
res = list(filter(lambda x: x%2==0, nums))
print(res)
returns: [22,44]
```

Maps

```
nums = [11, 22, 33, 44, 55]

result = list(map(lambda x: x+5, nums))
print(result)
```

13. Classes

13.1. Creation

A class is a blueprint for the creation of objects.

A class is formed with the keyword `class` and the classname followed by a colon. By tradition, the first letter of a classname is capitalized.

The class generally has a **constructor** which is designate with the, keyword `'__init__'` and contains the object data for each instance.

In the constuctor function, the keyword **self** refers to the oject being formed. Some people like to start the class data with on underscore to (e.g. `self._first`) remind people not to access it directly from outside the class.

```
class Employee:
    def __init__(self, first,last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.emai. = first+'.'+last+'@company.com'
    def fullname(self):
        return'{} {}'.format(self.first, self.last)
```

An **object** is initialized by calling the class with the initial data.

```
emp_1 = Employee('Bill', 'Bell', 50000)
emp_2 = Employee('Mike', 'Myers', 60000)
```

Class functions are called by the objectname, e.g.
`print (emp_1.fullname())`

===

Many languages have private variables for classes so that class data can never be accessed directly from outside. Python does not have private variables. For people regarding that as very important they change the name of the variable and introduces the ideas of getters and setters

```
self._first
getter
def first(self)
    return self.first
combined getter and setter
def first(self, f = None)
    if f: self._first = f
    return self._name
```

13.2. Class Variables

Class variables vs. Instance variables

Class variables are variables that are shared amount all instances of a class.

```
class Employee:
    num_of emp = 0
    raise_amount = 1.04
    def __init__(self, first,last, pay):
        self.first = first
        self.last = last
```

```

self.pay = pay
self.emai. = first+'.'+last+'@company.com'
Employee.num_of_emps += 1
def fullname(self):
    return'{} {}'.format(self.first, self.last)
def apply_raise(self)
self.pay = (self.pay * self.raise_amount)

```

The class variable can also be altered externally.

```
Employee.raise_amout = 1.05
```

```
----
```

```
emp_1.raise_amount = 1.06
```

will apply function to instance only

```
run: print(emp_1.__dict__)
```

before and after last function shows that raise_amount becomes an entry into emp_1 dictionary which it wasn't before.

The importance here is the instance data overrides the class data.

```
-----
```

The number of employees is a perfect example of a class variable and is never a part of any instance data.

```
print(Employee.num_of_emp)
```

13.3. Methods

regular methods

class methods

static methods

A static method is a method that does not rely on either class or instance data but is somehow related to class function.

Class method decorator

cls refers to class (as opposed to self) because class is used in another context.

```
class Employee:
```

```
def __init__(self, first,last, pay):
```

```
    self.first = first
```

```
    self.last = last
```

```
    self.pay = pay
```

```
    self.emai. = first+'.'+last+'@company.com'
```

```
def fullname(self):
```

```
    return'{} {}'.format(self.first, self.last)
```

```
@classmethod
```

```
def set_raise_amt(cls, amount):
```

```
    cls.raise _amt = amount
```

```
@classmethod
```

```
def from_string(cls, emp_str):
```

```
    first, last, pay = emp_str.split(-)
```

```
    return cls(first, last, pay)
```

Second class method above is used as an alternative construction and is executed as follows:

```
Employee.from.string(John-Doe-70000)
```

13.4. Inheritance

Inheritance is useful in that it allows us to inherit attributes and methods from a parent class.

Parent Class

```
class Employee:
    raise_amount = 1.04
    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        self.emai. = first+'.'+last+'@company.com'
    def fullname(self):
        return'{} {}'.format(self.first, self.last)
    def apply_raise(self):
        self.pay = (self.pay * self.raise_amount)
```

Child Classes

A Child class is declared with class keyword, classname, parent in parens followed by colon.

For both data and method programs looks for specified data and methods in its own class and if it doesn't find it, looks for it in its parent class.

```
class Developer(Employee):
    raise_amount = 1.10
    def __init__(self, first, last, pay, prog_lang):
        super().__init__(first, last, pay)
        self.prog_lang = prog_lang

class Manager(Employee):
    def __init__(self, first, last, pay, employees=None):
        super().__init__(first, last, pay)
        if employees is None:
            self.employees = []
        else:
            self.employees = employees
    def radd_emp(self, emp):
        if emp not in self.employees:
            self.employees.append(emp)
    def remove_emp(self, emp):
        if emp in self.employees:
            self.employees.remove(emp)
    def print_emps(self):
        for emp in self.employees:
            print(emp.fullname())
```

```
dev_1 = Employee('Bill', 'Bell', 50000, 'Python')
dev_2 = Employee('Mike', 'Myers', 60000, 'Java')
mgr_1 = Manager('Sue', 'Smith', 90000, [dev_1])
```

13.5. Class Example

```
class Employee():
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
```

```

        self.salary = salary

    def __repr__(self):
        return '{},{},{}'.format(self.name, self.age, self.salary)

e1 = Employee('Carl', 37, 70000)
e2 = Employee('Sarah', 29, 80000)
e3 = Employee('John', 43, 90000)
employees = [e1, e2, e3]

def e_sort(emp):
    return emp.salary

s_employees = sorted(employees, key=e_sort, reverse=True)
print(s_employees)

```

14. File I/O

14.1. File R/W

Text files can be opened in two ways.

1. `f = open('test1.txt', 'rt')`

2. `with open('test1.txt', 'rt') as f:`

The second way takes care of house keeping. It closes the file automatically when the block is closed or when any error is generated. Otherwise the file must be explicitly closed

open modes

`rt`

`wt`

`rb`

`wb`

`a` append

`r++` read and write

reading from open file

`c = f.read()` Reads entire file

`c = f.readline()` Reads line including newline

`c = f.readlines()` Reads every line including newlines

os module

`f.write(c)`

`f.name`

`f.open()`

`f.close()`

`f.mode()`

`f.closed()` returns True or False

`location = f.tell()` returns file pointer

`f.seek(location)` sets file at location

Copy or process text file

`with open('test1.txt', 'rt') as rf:`

`with open('test2.txt', 'wt') as wf:`

`for line in rf:`

`wf.write(line)`

`with open("buf.txt", "r") as file:`

`for line in file:`

`print(line,)`

`print(file.readlines())` # outputs list

Copy Binary File

`with open('photo1.jpg', 'rb') as rf:`

`with open('photo2.jpg', 'wb') as wf:`

`chunk_size= 4096`

`rf_chunk = rf.read(chunk_size)`

`while len(rf_chunk) > 0:`

`wf.write(rf_chunk)`

`rf_chunk = rf.read(chunk_size)`

In **Jupyter**, a file can be created as follows

`%%writefile test.txt`

First Line

Second Line

14.1.1. Try/Except

```
fname = input('Enter the file name: ')
try:
    fhand=open(fname)
except:
    print('File cannot be opened: 'fname)
    quit()
```

14.2. json

JSON = Java Script Object Notation

It is a data exchange format similar to XML

```
{
"name": "tom",          <name>Tom</name>
}
```

It differ from xml in that opening tag is not repeated and therefore a file becomes smaller.

differences with python

null instead of None

true, false not capitalized

In Python, the data becomes a dictionary

Key methods are:

import json

json.load(f) - loads json data from file

json.loads(s) - loads json data from string

json.dump(j,f) - writes json data to file

json.dump(j) - outputs json object as string

14.2.1. sample 1

```
import json
people_string = '''
{
    "people": [
        {
            "name": "John Smith",
            "phone": "615-555-7164",
            "emails": ["johnsmith@bogusemail.com", "john.smith@work-place.com"],
            "has_license": false
        },
        {
            "name": "Jane Doe",
            "phone": "560-555-5153",
            "emails": null,
            "has_license": true
        }
    ]
}
'''

data = json.loads(people_string)
print(data)

for person in data['people']:
    print(person)
    print(person['name'])
```

```
new_string = json.dumps(data, indent=2)
print(new_string)
```

Source: [import json](#) (6 September 2019 2:42 AM)

14.2.2. sample 2

```
{
  "title": "Gattaca",
  "release_year": 1997,
  "is_awesome": true,
  "won_oscar": false,
  "actors": ["Ethan Hawke", "Uma Thurman", "Alan Arkin"],
  "buget": null,
  "credits": {
    "director": "Andrew Niccol",
    "writer": "Andrew Niccol",
    "composer": "Michael Nyman",
    "cinematographer": "Slawomir Idziak"
  }
}
```

in this sample json packete all the data types are included
strings
ints
bolleans
list
another json object

14.3. Operating system

```
import os
```

To see all commands
`print (dir(os))`

Source: [print \(dir\(os\)\)](#) (19 August 2019 10:12 AM)

14.3.1. getcwd

Get current working directory

14.3.2. chdir

```
chdir('j:/temp')
```

Source: [os.chdir\('j:/temp'\)](#) (19 August 2019 10:13 AM)

14.3.3. listdir

```
print(os.listdir(path))
```

14.3.4. mkdir

```
mkdir(path)
mkdirs(path)
    will create deep subdirectory path
```

14.3.5. rmdir

```
os.rmdir(path)
os.removedirs()
    will remove subdirectory path
```

14.3.6. rename

```
os.rename(old, new)
```

14.3.7. stat

```
st_mode
st_ino
st_dev
st_nlink
st_uid
st_gid
st_size
st_atime
st_mtime
st_ctime
```

```
to get last file modification time
mod_time = os.stat('demo.txt').st_mtime
print(datetime.fromtimestamp(mod_time))
```

Source: [st_mode](#) (19 August 2019 10:40 AM)

14.3.8. dirpath

```
for dirpath, dirnames, filenames in os.walk(dir)
    print('Current path: ', dirpath)
    print('Directories: ', dirnames)
    print('Files: ', filenames)
    print()
```

14.3.9. walk

```
os.walk(dir)
```

provides directory tree of everything below path

14.3.10. environ

`os.environ.get('HOME')`

14.3.11. path.exist

`os.path.exist('dir')`

14.3.12. path.splitext

`os.path.splitext()`
returns path and file extension

14.3.13. path.isdir

`path.isdir('path')`
returns True or False

14.3.14. path.isfile

`path.isfile('path')`
returns True or False

15. Librarys

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

rfn = 'j:\Python\Mar19\Olympics\Olympics.csv'
df = pd.read_csv(rfn, skiprows=4)
```

15.1. Importing styles

example module = my_module
(this is contents of file in same directory or on path)

```
test = 'Test String'

def find_index(to_search, target):
    """Find the index of a value in a sequence"""
    for i, value in enumerate(to_search):
        if(value == target):
            return i
    return -1
----
```

```
if __name__ == '__main__': main()
C:\wp\Python37\lib\site-packages
```

working program import styles

```
courses = ['History', 'Math', 'Physics', 'CompSci']
```

import my_module

```
index = my_module.find_index(courses, 'Math')
print(index)    returns 1
```

import my_module as mm

```
index = mm.find_index(courses, 'Math')
```

from my_module import find_index

```
find_index(courses, 'Math')
```

from my_module import find_index as fi

```
fi(courses, 'Math')
```

from my_module import find_index, test

Can import other things separated with commas

Import File Location

```
method 1:
add to top of file
import sys
sys.path.append('j:/Python_Modules')
```

change python path variable to environment

15.1.1. Path

Documented in "Import modules..." Windows at 11:40

```
PATHONPATH="LOCATION"    No spaces anywhere  
>>>import sys    11:40  
>>>sys.path
```

Start
right click on Computer
goto properties
Advance system settings
Envirement Variables
Create New Environment Variable
click new
Name = PYTHONPATH
Location=J:\python
ok

then import should work

15.2. Standard Library

[datetime](#)

[Calendar](#)

[Operating System](#)

[Regular Expressions](#)

[subprocess](#)

15.3. Time&Date

15.3.1. Time

```
import time  
now=time.time()    # returns number of seconds since 1/1/1970  
round(now,n)  
endTime-startTime  
time.sleep(n)    # pauses n seconds
```

15.3.2. DateTime

from datetime import datetime

```
days = ('Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat')
```

```
now = datetime.now()
print(now) Returns: 2019-09-03 09:39:17.315523
```

```
x = now.weekday()
print(f"Today's weekday # is: {x}")
print(f"Today is {days[x]}")
```

strftime

The function strftime returns the following format codes

```
# Date Formatting
# %y/%Y = year      e.g. 19 vs. 2019
# %a/%A = weekday  e.g. Tuesday vs. Tue
# %b/%B = month     e.g. September, vs Sep
# %d = day of month - 2 digit
```

```
# Time formatting
# %I/%H = 12/24 hour
# %M = minute
# %S = second
# %p = locale's am/pm
```

```
print(now.strftime("The current year is %B"))
print(now.strftime("The current day of month %d"))
print(now.strftime("Local date and time: %c"))
=====
```

15.3.3. Calendar

import calendar

```
print(calendar.weekheader(3))
# Returns: Mon Tue Wed Thu Fri Sat Sun
```

```
print(calendar.firstweekday())
# Returns 0
```

```
print(calendar.month(2019, 9))
September 2019
Mo Tu We Th Fr Sa Su
```

```
1
2 3 4 5 6 7 8
9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30
```

Note: The default format is European with Sunday on the right. To print calendar with Sunday on left, the following code must be run first

```
calendar.setfirstweekday(6)
```

```
print(calendar.monthcalendar(2019,9))
Returns week lists of days, i.e. [[0, 0, 0, 0, 0, 0, 1], [2, 3, 4, 5, 6, 7, 8],
```

```
print(calendar.calendar(2019))
Returns calendar of year
```

```
x = calendar.isleap(2020)
print(x) Returns: True
```

```
day_of_the_week = Calendar.weekday(300, 3, 8)
print(day_of_the_week) Returns 0 for Monday
```

Source: print(day_of_the_week) (4 September 2019 3:09 AM)

```
# how_many_leap_days = calendar.leapdays(2000, 3000)
Returns 243
```

15.4. Random

Random library has many functions

```
import random
```

```
shuffles in place
shuffle(mylist)
mylist
```

```
randint(0,100)
returns random int between 0 and 100
```

15.5. Regular Expressions

Regular Expressions lib is used to search for textual patterns in textual materials, eg phone numbers, email addresses etc.

Regex Pattern
telephone number: `r"(/d/d/d)-/d/d/d-/d/d/d/d"`

Typical useage
`import re`

```
text="The agent's phone number is 619-840-5544"
pattern - "phone"
match=re.search(pattern, text)
match.span() (12,17)
match.start()
math.end()
```

```
for multiple hits
matches = re.findall('phone', text)
for match in re.finditer('phone', text):
    print(match.span())
```

Python Regular Expression Quick Guide

```
.    Matches any character
\d   Digit (0-9)
\D   Not a digit
\w   Word Character (a-z, A-Z, 0-9, _)
\W   Not a word character
\s   Matches whitespace
\S   Matches any non-whitespace character
[aeiou] Matches a single character in the listed set
```

[^XYZ] Matches a single character not in the listed set
[a-z0-9] The set of characters can include a range
(Indicates where string extraction is to start
) Indicates where string extraction is to end

Boundaries

\b - Word Boundary
\b - Not a word Boundary
^ - Beginning of a string
\$ - End of a string

Quantifiers:

* - 0 or more
*? - 0 or 1 (non-greedy)
+ - 1 or more
+? = 0 or 1 (non greedy)
? - 0 or 1
{3} - Exact number
{3,4} - Range of Numbers
[3-7] - Range of numbers

Sample Regexs

email address:
[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+[a-zA-Z0-9-.]+

Sample Operations

```
import re
```

```
str2 = 'Send an email from this@email.com to test@user.com'
result = re.search('^Send', str2)
print(result)
returns: <re.Match object; span=(0, 4), match='Send'>
```

```
str = 'abcdefg Bill 44 Bell 840-5544'
result = re.findall('[0-9]+', str)
print(result)
returns: ['44', '840', '5544']
```

```
str1 = 'abcdefg Bill abc 44 Bell 840-5544'
result = re.findall('[0-9]+.[0-9]+', str1)
print(result)
returns: ['840-5544']
```

```
str2 = 'Send an email from this@email.com to test@user.com'
result = re.findall('\S+@\S+', str2)
print(result)
returns: ['this@email.com', 'test@user.com']
```

```
text = open('shakespear.txt')
for line in text:
    line = line.rstrip()
    if re.search('^A.+$', line) :
        print(line)
#returns: All lines starting with capital A
```

15.5.1. Misc

[Python.org Documentation](#)
[w3schools.com](#)
[Tutorials point](#)

[Pythex - regular expression testing](#)

15.5.2. Functions

15.5.2.1. re.search

`re.search(pattern, string, flags=0)`

returns first match found or none

```
txt = 'The rain in Spain'
x = re.search('/s', txt)
y = re.search('France', txt)
```

```
x = 3      (first white space)
y = None
```

15.5.2.2. re.findall

`re.findall(pattern, string, flags=0)`

Return all non-overlapping matches of *pattern* in *string*, as a list of strings. The *string* is scanned left-to-right, and matches are returned in the order found. If one or more groups are present in the pattern, return a list of groups; this will be a list of tuples if the pattern has more than one group. Empty matches are included in the result.

Changed in version 3.7: Non-empty matches can now start just after a previous empty match.

Source: [re — Regular expression operations — Python 3.7.4 documentation](#) (27 August 2019 1:04 PM)

15.5.2.3. re.compile

`compile(pattern, flags=0)`

Compile a regular expression pattern, returning a Pattern object.

15.5.2.4. re,copy_reg

15.5.2.5. re.error

15.5.2.6. re.escape

15.5.2.7. re.finditer

`finditer(pattern, string, flags=0)`

Return an iterator over all non-overlapping matches in the string. For each match, the iterator returns a Match object.

Empty matches are included in the result.

15.5.2.8. re.match

`match(pattern, string, flags=0)`

Try to apply the pattern at the start of the string, returning a Match object, or None if no match was found.

15.5.2.9. re.purge

15.5.2.10. re.split

`re.split(pattern, string, maxsplit=0, flag=0)`

```
txt = 'The rain in Spain'
x = re.split('/s', txt)
y = re.split('/s', txt, 1)
print(x)
print(y)
```

```
x = ['The', 'rain', 'in', 'Spain']
y = ['The', 'rain in Spain']
```

15.5.2.11. re.sre_compile

15.5.2.12. re.sub

`sub(pattern, repl, string, count=0, flags=0)`
replaces one string with another

```
txt = 'The rain in Spain'
x = re.sub('/s', '9' txt)
y = re.sub('/s', '9' txt, 2)
print(x)
print(y)
```

```
x = 'The9rain9in'9Spain'
y = 'The9rain9 in Spain'
```

Return the string obtained by replacing the leftmost non-overlapping occurrences of the pattern in string by the replacement repl. repl can be either a string or a callable; if a string, backslash escapes in it are processed. If it is a callable, it's passed the Match object and must return a replacement string to be used.

15.5.2.13. re.subn

`subn(pattern, repl, string, count=0, flags=0)`

Return a 2-tuple containing (new_string, number).

new_string is the string obtained by replacing the leftmost non-overlapping occurrences of the pattern in the source string by the replacement repl. number is the number of substitutions that were made. repl can be either a string or a callable; if a string, backslash escapes in it are processed.

If it is a callable, it's passed the Match object and must return a replacement string to be used.

15.5.2.14. re.template

15.5.3. Examples

```
import re
s="The ape was at the apex"
if re.search("ape", s):
    print("found")
# found
all = re.findall("ape", s)
for i in all:
    print(i)
#ape
#ape
#Square brackets match any character included
s="Cat rat mat fat pat"
all=re.findall("[crmf]at", s)
for i in all:
    print(i)
#finds all but Cat because of capital
#finding characters within range
all = re.findall("[c-mC-M]at", s)
for i in all:
    print(i)
#finds Cat, mat, fat
# ^ means find any char but
all = re.findall("[^Cr]at", s)
for i in all:
    print(i)
#finds all except Cat & rat
```

Source: [import re](#) (26 October 2019 6:10 AM)

15.6. Matplotlib

[Matplotlib Torii Series \(Corey Schafer\)](#)

[Online Code](#)

[Matplotlib Documentation](#)

<https://www.youtube.com/watch?v=a9UrKTVeeZA>

```
import pandas as pd
from matplotlib import pyplot as plt
```

Sample plot

```
x = [1,2,3]
y = [1,4,9]
z = [10,5,0]
plt.plot(x,y, label='This is Y')
plt.plot(x,z, label = 'This is Z')
plt.title('Test Plot')
plt.xlabel('x')
plt.ylabel('Y and Z')
plt.show()
```

Options:

Saving plot
`plt.savefig('plot.png')`

Styles

1. `print(plt.style.available)`
2. `plt.style.use('fivethirtyeight')`

A listing of available styles shown by #1 above. When one is chosen #2 shows how it is implemented. The various styles include things like line width and color, grids etc. Otherwise they can be set as indicated below.

```
plt.plot(options)
format string = 'k_'
color = 'k', '#5a7d9a'
marker = 'o'
linewidth = 3
linestyle = '--'
```

```
plt.grid(True)
```

15.7. Internet

Libraries

socket
urllib

Source: [are ports and sockets the same thing - Google Search](#) (27 August 2019 2:46 AM)

15.7.1. Misc

response codes from website

200 good
400 bad request
403 forbidden
404 not found
500s server errors

15.7.2. Sockets

A **socket** is one endpoint of a two-way communication link between two programs running on the network. A **socket** is bound to a **port** number so that the TCP layer can identify the application that data is destined to be sent to. An endpoint is a combination of an IP address and a **port** number.

A low level connection to a website can be accomplished with sockets, but that is not likely to be done today

```
import socket
```

```
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)
```

```

while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode(),end="")

mysock.close()

```

Get is used to request a document from a web server. The protocol demands an empty line follows. Encode and decode are used to ensure same character set is used.

Returns:

```

Last-Modified: Sat, 13 May 2017 11:22:22 GMT
ETag: "a7-54f6609245537"
Accept-Ranges: bytes
Content-Length: 167
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: text/plain
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief

```

Source: [Last-Modified: Sat, 13 May 2017 11:22:22 GMT](#) (27 August 2019 3:09 AM)

15.7.3. urllib

urllib makes things easier by doing most of the socket work.

urllib contains 4 modules

request: open urls

response: (used internally)

error: request exceptions

parse: useful url functions

robotparser: robots.txt files

```

>url = "https://youtube.com/watch?v=EuC-yVzHhMI&t=tm56s"
>resp = request.urlopen(url)
>resp.code > 200
>html = resp.read().decode("utf-8")
>html[:500]

```

Documents

```
>>>from urllib import request
```

```

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')
for line in fhand:
    print(line.decode().strip())

```

Binary files

For small files (i.e. less than memory size)

```
import urllib.request, urllib.parse, urllib.error
```

```
img = request.urlopen('http://data.pr4e.org/cover3.jpg').read()
fhand = open('cover3.jpg', 'wb')
fhand.write(img)
fhand.close()
```

for larger files

```
import urllib.request, urllib.parse, urllib.error.
```

```
img = request.urlopen('http://data.pr4e.org/cover3.jpg')
fhand = open('cover3.jpg', 'wb')
size = 0
while True:
    info = img.read(100000)
    if len(info) < 1: break
    size = size + len(info)
    fhand.write(info)

print(size, 'characters copied.')
fhand.close()
```

15.7.4. requests

```
pip install requests
import requests
```

a **GET** request, is used to retrieve data.

```
page = requests.get("http://www.google.com")
page.status_code
page.content
```

```
from bs4 import BeautifulSoup
```

```
soup = BeautifulSoup(page.content, 'html.parser')
print(soup.prettify())
```

15.7.5. Web Scraping

15.7.5.1. requests

Requests opens a file from the web and it is not suitable in opening a local file. To do that, e.g.

```
with open('simple.html') as fp:
    soup = BeautifulSoup(fp, 'lxml')
```

```
fp = requests.get('http://google.com")
soup = BeautifulSoup(fp.content, 'lxml')
print(soup.prettify())
```

[requests documentation](#)

Example:

```
import requests
```

```
r=requests.get("http://google.com")
```

```
print(r.status_code)
```

#The output provides the status code for the url. For a success full attempt, the result is 200

```
print(r.headers)
#The following attribute returns a python dictionary containing key-value pair of the headers

print(r.text)
#This command prints out the response content from the server or Static Source Code.

print(r.encoding)
r.encoding = 'utf-8'
#Requests library also allows you to see or change the encoding of the response content.
```

response codes from website

- 200 good
- 400 bad request
- 403 forbidden
- 404 not found
- 500s server errors

=====

After fetching the web page it can be viewed with BeautifulSoup as follows:

```
from bs4 import BeautifulSoup as bs
```

```
soup = bs(r.content)
print(soup)
print(soup.prettify())
```

===

Note: A latter library(requests-HTML) combines the functions included in requests and beautiful soup, but it doesn't seem to have a prettify functions which makes later coding harder.

15.7.5.2. Beautiful Soup

[Comprehensive Python BeautifulSoup Web Scraping Tutorial!](#)
[Beautiful Soup Documentation](#)

[HTML Crash Course](#)
[CSS Crash Course](#)
[CSS Selector Reference](#)

```
import requests
from bs4 import BeautifulSoup as bs
```

To open and examine an imported file with requests lib

```
html_file = requests.get('http://google.com')
soup = bs(html_file.content, 'lxml')
print(soup.prettify())
```

find (finds first target only) and find_all

```
first_header = soup.find("h2")
first_header = soup.find_all("h2") #returns a list
headers = soup.find("h1", "h2")
```

finds can be nested, e.g.

```
body = soup.find("body")
div = body.find("div")
```

Search for strings

Complete strings can be searched for. Smaller targets can be searched for by using the re library, e.g.

```
s = re.compile([H,h][12])
s2 = soup.find_all(s)
```

Paths

15.7.5.3. Requests-HTML

[Requests-HTML Documentation](#)

[Requests-HTML: A Python Library For Scraping The Web](#)

[Web Scraping with Requests-HTML \(CS\)](#)

[Web scraping Cheat Sheet](#)

Historically, html was fetched by urllib and then parsed with "Beautiful Soup." Today this new lib does both

```
from requests_html import HTML
```

Sample program syntax

```
'''Returns all links on a page'''
```

```
>session = HTMLSession()
```

```
>r = session.get('https://python.org/')
```

```
>links = r.html.links
```

```
>for link in links:
```

```
>    print(link)
```

```
from requests_html import HTML
```

```
with open('simple.html') as html_file:
```

```
    source = html_file.read()
```

```
    html = HTML(html=source)
```

```
articles=html.find('div.article')
```

```
for article in articles:
```

```
    headline = article.find('h2', first=True).text
```

```
    summary = article.find('p', first=True).text
```

```
    print(headline)
```

```
    print(summary)
```

```
r = session.get(url)
```

```
r returns error code (200 or less success)
```

```
r.html.links - returns links
```

```
r.html.absolute_links
```

With crome,you can use web inspector to find the line in the code where the element is accessed.

```
r.html.find('.headline') CSS selector
```

```
r.html.find('.headline', first = True)
```

True will give first find only

```
headline = r.html.find('.headline', first = True)
```

```
headline.text
```

Provides content

```
headline.html
```

Provides entire line

apparent_encoding
close
connection
contentr.
content_consumed
Elapseddr
encoding
headers
history
html
is_permanent_redirect
is_redirect
iter+con
tent
iter_lines
json
links
next
ok
raise_for_status
raw
reason
request
session
status_code
text
url

15.7.5.3.1. Functions

15.8. subprocess

```
import subprocess

#to run a op specific command
#additional arguments can be part of string
#subprocess.run('dir', shell=True) or
#subprocess.run(['dir', ' -b']) #does not work
#subprocess.run('dir j:\\jb', shell=True)

#to copy into variable
#capture_output > to prevent screen display===
p1 = subprocess.run('dir j:\\jb', shell=True, capture_output=True)
#print(p1) #returns listing of bytes
print(p1.stdout.decode()) #returns formatted text
print(p1.returncode) #returns 0 if successful
print(p1.stderr)

#to put in file
with open('output.txt', 'w') as f:
    subprocess.run('dir J:\\jb', shell=True, stdout=f, text=True)
```

15.9. Beautiful Soup4

<https://www.dataquest.io/blog/web-scraping-tutorial-python/>

15.10. numpy

user guide:

<https://docs.scipy.org/doc/numpy/>

Course:

<https://www.dataquest.io/course/python-for-data-science-intermediate/>

johnbi@excite fido49jo

=====

```
# https://www.youtube.com/watch?v=GB9ByFAIAH4
https://github.com/joeyajames/Python/tree/master/Numpy
```

```
pip install numpy
import numpy as np
import sys
```

```
# The basics
# a = np.array([1, 2, 3], dtype='int8')
# b = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
```

```
# # Print Matix
# print(a)
# print(b)
```

```
# # Get Dimentsion
# print("Dimention", a.ndim)
```

```
# # Get type/size
# print("Type", b.dtype, "Size ", b.itemsize, "bytes")
```

```
# # Get Shape
# print(a.shape)
# print(b.shape)
```

```
# # Get Type
# print(a.dtype)
# print(b.dtype)
```

```
# # Get Size
# print(a.itemsize) # 1
# print(b.itemsize) # 8
# print(a.size)     # 3
# print(b.size)     # 6
```

```
# c = np.array([[1, 2, 3, 4, 5, 6, 7], [8, 9, 10, 11, 12, 13, 14]])
# print(c)
# print(c.shape)
```

```
# Get specific elemnts, rows, columns, etc.
```

```

# # Get specific element (r,c)
# print(c[1, 5]) # 13
# print(c[1, -2]) # 13

# Get specific row
# print(c[0, :]) # [1 2 3 4 5 6 7]

# # Get specific column
# print(c[:, 2]) # [3 10]

# =====
# # Initializing Different Types of Arrays

# # All zero arrays
# d = np.zeros(5)
# e = np.zeros((2, 3))
# print(d) # [0,0,0,0,0]
# print(e)

# # Any other number
# f = np.full((2, 2), 99)
# print(f)

# g = np.full_like(e, 4)
# print(g)

# # random decimal numbers
# h = np.random.rand(3, 2)
# print(h)

# # random integer values
# i = np.random.randint(7, size=(3, 3))
# j = np.random.randint(3, 7, size=(3, 3))
# print(j)

# # To copy arrays
# k = j.copy()

# # Mathematics
# l = np.array([1, 2, 3, 4])
# print(l+2) # [2 4 5 6]
# print(l-2)
# print(l*2)
# print(l/2)
# print(l**2)

# Statistics
# stats = np.array([[1, 2, 3], [4, 5, 6]])
# m1 = np.min(stats)
# m2 = np.max(stats)
# m3 = np.sum(stats) # 21
# m4 = np.sum(stats, axis=1) # [6 15]
# m5 = np.sum(stats, axis=0) # [5 7 9]
# m6 = np.mean(stats, axis=1) # [2. 5.]
# m7 = np.var(stats, axis=1) # variance
# m8 = np.std(stats, axis=1) # standard deviation
# print(m7)
# print(m8)

```

```

# # Miscellaneous
# # Reorganizing Arrays
# before = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
# print(before)
# after = before.reshape(4, 2)
# print(after)
# after = before.reshape(8, 1)
# print(after)

# # stacking
# v1 = np.array([1, 2, 3, 4])
# v2 = np.array([5, 6, 7, 8])

# s1 = np.vstack([v1, v2])
# print(s1)

# # Load text from a file
# filedata = np.genfromtxt('data.txt', delimiter=',')

```

15.11. Pandas

```

Last,First,email,salary,code
Bell,Bill,BillBell@yahoo.com,50000,HTML;Python
Doe,Jane,JaneDoe@email.com,60000,C++
Doe,John,JohnDoe@email.com,70000,SQL;Python

```

```
import pandas as pd
```

```

df = pd.read_csv('filename')
print(df.head(n))
print(df.tail(n))

```

Dataframe Info

```

df.shape      # prints number rows and columns
df.info()     # prints column names
df.columns

```

display rows

```

df.head(n)    # default 5
df.tail(n)
df.loc[0]
pd.set_options('display.max_rows')

```

Read headers

```

df.columns
df.column[0:5]
df['header']
df['header'].value_counts()
df[['header1', 'header2']]
pd.set_option('display.max_columns')
Read each column[0:5])

```

Read each row

```

df.iloc[0]
df.iloc[[0,1], n] # n is column wanted
df.loc[[0,1], ['header', 'header2']]

```

Read a specific location (R,C)

```
df.loc[n, 'header']
df.loc[[n1, n2], 'header']
```

Index operations

```
df.set_index('header, inplace = True)
df.reset(index(inplace = True)
```

Filtering

```
filt = (df['header'] == 'Target')
filt = (df['First'] == 'Jane')
or: filt = (df['First'] == 'Jane') | (df['First'] == 'Bill')
and: filt = (df['header'] == 'Target')
df[filt] or
df.loc[filt]
df.loc[filt, 'email']
df.loc[~filt, 'email']
--
filt = df['code'].str.contains('Python')
df.loc[filt]
```

Updating Rows and Columns

```
df.columns
df.columns = ['firstname', 'secondname', etc]
df.columns = x.lower() for x in df.columns
df.rename(columns={'first': 'first_name'}, inplace=True)
- Rows
df.loc[2] = ['value', 'value' ...]
df.loc[2, 'last'] = 'Smith'
```

Adding and removing rows and columns

```
- Columns
df['full_name'] = df['first'] + ' ' + df['last']
df.drop(column=['first', 'last'], inplace=True)
- Rows
df.append
df.drop(index=4)
```

Sorting

```
df.sort_values(by='last')
df.sort_values(by='last', 'first')
df.sort_index()
df.nlargest('code')
```

Grouping and Aggregating Data

```
df[salary].median()
df.describe()
df['field'].value_counts()
df['field'].value_counts(normalize=True) # gives per cent
-- grouping
country=df.groupby(['country'])
country['field'].value_counts()
```

Cleaning data

```
- Dropping rows
df.dropna('index', how='any') # default
'index' or 'column'
any, all, subset=['headers']
inplace=True to do for real
```

```
str.contains()
str.startswith()
str.isnumeric()
```

15.12. webbrowser

```
import webbrowser
webbrowser.open('https://automatetheboringstuff.com')
```

15.13. pyautogui

<https://pyautogui.readthedocs.io/>

```
import pyautogui
The source is available on: https://github.com/asweigart/pyautogui
pip install psautogui
```

Mouse methods

```
pyautogui.size()
pyautogui.position()
pyautogui.moveTo(10.10,duration=1.5)
pyautogui.moveRel(200,0,duration=)
pyautogui.click(x,y)
pyautogui.doubleClick(x,y)
pyautoguie.rightClick(x,y)
pyautogui.middleClick(x,y)
pyautogui.dragRel(distance, distance, duration)
failsafe to 0,0
pyautogui.displayMousePostion() #position and color
```

Keyboard Methods

```
pyautogui.press() will press a single key.pyautogui.hotkey() combinations like Ctrl-O.
pyautogui.hotkey('ctrl', 'o')
pyautogui.typewrite("this text", interval=0.2)

pyautogui.click('a', left, 'B', 'f11', 'volumnmute' )
pyautogui.KEYBOARD_KEYS)
```

Image methods

```
pyautoguie.screenshot('c:\\screenshot_example.png')
pyautoguie.locateOnScreen('c:\\calc7key')
pyautoguie.locateCenterOnScreen)
```

15.13.1. samples

```
for i in range(10):
    py.moveTo(100,100,duration=0.25)
    py.moveTo(200,100,duration=0.25)
    py.moveTo(200,200,duration=0.25)
    py.moveTo(100,200,duration=0.25)

pyautogui.click(200, 250, button='right')
pyautogui.mouseDown()
pyautogui.mouseUp()
pyautogui.doubleClick()

import pyautogui as py
import time
```

```

time.sleep(5)
py.click() # Click to make the window active.
distance = 300
change = 20
while distance > 0:
    py.drag(distance, 0, duration=0.2) # Move right.
    distance = distance - change
    py.drag(0, distance, duration=0.2) # Move down.
    py.drag(-distance, 0, duration=0.2) # Move left.
    distance = distance - change
    py.drag(0, -distance, duration=0.2) # Move up.

```

15.14. Selenium

UFPRKJJCW6LFZHEOYJ5N BG24DFH4GPKXO3JEKUQ4VCOM6FE3GJFA
<https://selenium-python.readthedocs.org>

pip install selenium-python
 DOWNLOAD chrome driver from selenium site

```

from selenium import webdriver
driver = webdriver.chrome(path to chrome driver)

```

15.15. AutoIT

To get coordinates on autoit window info program, set coordinate move to 'Window'.
 Drag the 'Finder Tool' to operative location and click.
 Expand the app to click on "Mouse"
 Position shows the x y coordinates.

```

#RequireAdmin
Run('path/filename' '')
AutoItSetOption('MouseCoordMode', 0)
WinWait('Classic Shell Setup')
WinActivate('Classic Shell Setup')
MouseClick('primary', 668, 549, 1, 0)
=====
#10
Methods of clicking/checking/unchecking things

```

```

WinActivated(...)
MouseClick('primary', x,y,1,0)
ControlClick('CCleaner', "", Button7)

```

```
ControlCommand('CCleaner', '', 'Button7', 'Uncheck')
```

Control are ordered top to bottom
 You can Tab to right control and then click space to selected it
 i.e.

```

Send('(TAB(Tab)(SPACE)')
=====

```

Methods of typing Text into box

1.

```
Run('Notepad', "")
WinWait('Untitled - Notepad')
WinActivate('Untitled - Notepad')
Send('This is a test', 5)
The number sets the speed of typing
Special Keys can also be send (see Help)
```

2. Using clip board

```
ClipPut("Here is second line of text")
WinActivate('Untitled - Notepad')
Send('^v')
```

16. Functional Programming

```
def calc(f, x, y):  
    return f(x, y)  
  
def add(x, y):  
    return x + y  
  
def sub(x, y):  
    return x - y  
  
a = calc(add, 10, 20)  
b = calc(sub, 20, 15)  
print(f'a = {a}, b = {b}')  
=====
```

17. Database

SQLite- file based database

View>Tool Windows>database (not in free edition)

17.1. Install

<http://sqlite.org>
Under windows download

sqlite-ddl-win64...
sqlite-tools

Unzip both into the same directory

note: the tools will go into a subdirectory but they must be placed into the same mother directory.

Then the sqlite directory should be located on the path

17.2. Commands

Command line commands

open file: `sqlite3`
results in "`sqlite>`" prompt
to exit: `".quit"`

Create database

Database can be created either in this directory or elsewhere with this command:

`sqlite3 "mydatabase.db"`

`sqlite> .databases` Will list all databases

Create table(e.g. students)

`sqlite> create table students (fields);`

`sqlite> .tables` Will list all tables created

`sqlite> .schema students` Will show table format

[ProgrammingKnowledge series of tutorials](#)

[SQLite video tutorial \(Derek Banas\)](#)

18. Scratch

18.1. Spider

```
import csv
from requests_html import HTML, HTMLSession

csv_file = open('cms_scrape.csv', 'w')
csv_writer = csv.writer(csv_file)
csv_writer.writerow(['headline', 'summary', 'video'])
wp = open('CoreySchafer.txt', 'w')

def fileProcess(filename):
    i = 0
    r = session.get(filename)
    articles=r.html.find('article')

    for article in articles:
        x=len(articles)
        headline = article.find('.entry-title-link', first=True).text
        summary = article.find('.entry-content p', first=True).text

        try:
            vid_src = article.find('iframe', first=True).attrs['src']
        except Exception:
            continue

        vid_id = vid_src.split('/')[4]
        vid_id = vid_id.split('?')[0]
        yt_link = f'https://youtube.com/watch?v={vid_id}'

        wp.write(headline + '\n')
        wp.write(summary + '\n')
        wp.write(yt_link + '\n\n')

        i += 1
        csv_writer.writerow([headline, summary, yt_link])
        csv_writer.writerow(headline)
    return i

session = HTMLSession()
fileProcess('https://coreyms.com/')
sum = 0
for x in range(2, 17):
    filename = f'https://coreyms.com/page/{x}'
    print(filename)
    i = fileProcess(filename)
    sum = sum + i

print(f'{sum} articles processed' )
wp.close()
csv_file.close()
```

```

# from requests_html import HTML

# with open('simple.html') as html_file:
#     source = html_file.read()
#     html = HTML(html=source)

# # prints contents of file in text or html
# # print(html.html)
# # print(html.text)

# match = html.find("title", first=True)
# print(match.text)

# articles = html.find('div.article')
# for article in articles:
#     headline = article.find('h2', first=True).text
#     summary = article.find('p', first=True).text
#     print(headline)
#     print(summary)
#     print()

```

18.2. mac_changer

```

exam = "eth1"
ip = "11:22:38:44:55:66"

def is_interface_valid():
    mylist = []
    with open('ifconfig.txt', 'rt') as rf:
        for s in rf:
            x= s.find('flag')
            if x > 0:
                x = s.find(':')
                s= s[0:x]
                mylist.append(s)
    if exam in mylist:
        print("Thats a good one")
        return 0
    else:
        print("You did not enter a useable interface")
        if(len(mylist)) == 0:
            print("You need to enter something")
        else:
            print("Your choices are ")
            for i in range(len(mylist)):
                print(mylist[i])

def is_ip_valid(ip):
    error = 0
    if len(ip) != 17:
        error = -1
    x = re.match("(?:[0-9a-fA-F]:?){12}", ip)

```

```

if type(x) == type(None):
    error = -1
if(error == -1):
    print("Enter a valid MAC address formatted:")
    print("xx:xx:xx:xx:xx:xx")

is_interface_valid()
ip_valid = is_ip_valid(ip)
# print("Returned result = " + result)

```

Source: [exam = "eth1"](#) (8 November 2019 5:20 AM)

18.3. mac_changer1

```

import subprocess
import optparse

def get_arguments():
    parser = optparse.OptionParser()
    parser.add_option("-i", dest="interface", help="enter Interface")
    parser.add_option("-m", dest="new_mac", help="enter MAC address")
    (options, arguments) = parser.parse_args()
    return(options)

def change_mac(interface, new_mac):
    print("[+] Changing MAC address for " + interface + " to " + new_mac)
    print("Interface = " + interface)
    subprocess.call(["ifconfig", interface, "down"])
    subprocess.call(["ifconfig", "hw", "ether", new_mac])
    subprocess.call(["ifconfig", interface, "up"])

options = get_arguments()
change_mac(options.interface, options.new_mac)

```

18.4. mac_changer2

```

#!/usr/bin/env python3

import subprocess
import optparse

def get_arguments():
    parser = optparse.OptionParser()
    parser.add_option("-i", dest="interface", help="enter Interface")
    parser.add_option("-m", dest="new_mac", help="enter MAC address")
    (options, arguments) = parser.parse_args()
    return(options)

def change_mac(interface, new_mac):
    print("[+] Changing MAC address for " + interface + " to " + new_mac)
    subprocess.call("ifconfig " + interface + " down", shell=True)
    subprocess.call("ifconfig " + interface + " hw ether " + new_mac, shell=True)
    subprocess.call("ifconfig " + interface + " up", shell = True)

```

```

def get_current_mac(interface):
    ifconfig_result = subprocess.check_output(["ifconfig", Interface])
    mac_address_search_result(r"\w\w:\w\w:\w\w:\w\w:\w\w:\w\w", ifconfig_result)
    if mac_address_search_result:
        return mac_address_search_result.group(0)
    else:
        print("[-] Could not read MAC address.")

options = get_arguments()

current_mac = get_current_mac(options.interface)
print("Current MAC = " + str(current_mac))

change_mac(options.interface, options.new_mac)
current_mac = get_current_mac(options.interface)
if(current_mac == options.new_mac):
    print("[+] MAC address was successfully changed to " + current_mac)
else:
    print("[-] MAC address did not get changed.")

```